Plugging into Standard Methods

The notion of hooks extends beyond specialized methods like Class#inherited or Module#method_added. Because most operations in Ruby are just regular methods, you can easily twist them into improvised Hook Methods.

For example, in *Hook Methods*, on page 157, you learned how to override Module#included to execute code when a module is included. But you can also plug into the same event, so to speak, from the other side: because you include a module with the include method, instead of overriding Module#included, you can override Module#include itself.

For example:

```
module M; end

class C
  def self.include(*modules)
    puts "Called: C.include(#{modules})"
    super
  end

  include M
end
```

```
Called: C.include(M)
```

There is an important difference between overriding Module#included and overriding Module#include. Module#included exists solely to be used as a Hook Method, and its default implementation is empty. But Module#include has some real work to do: it must actually include the module. That's why our hook's code also should call the base implementation of Module#include through super. If you forget super, you'll still catch the event, but you won't include the module anymore.

As an alternative to overriding, you can turn a regular method into a Hook Method by using an *Around Alias (134)*. You can find an example of this technique in *The Thor Example*, on page 133.

The VCR Example

VCR is a gem that records and replays HTTP calls. The Request class in VCR includes a Normalizers::Body module:

```
module VCR
  class Request #...
    include Normalizers::Body
    #...
```

The Body module defines methods that deal with an HTTP message body, such as body_from. After the include, those methods become class methods on Request. Yes, you read that right: Request is gaining new class methods by including